ReviveIQ

IMPLEMENTATION REPORT · V1.0

Turn dead leads into recovered revenue.

Full architecture, scoring model, AI agent design, database schema, and phased build plan for the ReviveIQ platform.

DATE	VERSION	STATUS	AUTHOR
April 17, 2026	1.0 — Draft	Pre-build	ReviveIQ Team

Table of Contents

BUILD

01 — EXECUTIVE SUMMARY

What ReviveIQ does and why it matters.

The one-page version of the entire product, strategy, and plan.

1,000+
Dead leads in the average sales database

3–5×
Higher conversion on warm vs cold outreach

₹0
Extra spend to recover leads already acquired

The problem

Every sales team accumulates hundreds of leads who went cold — budget constraints, wrong timing, team restructuring. Every few months someone dumps the entire list on the calling team with no priority, no context, and no strategy. Agents call cold, in random order. Most leads are never contacted at all. Revenue quietly leaks.

What ReviveIQ does

ReviveIQ connects to your CRM, pulls all inactive/dead leads, and runs each one through an AI scoring pipeline that combines CRM history with live web intelligence (funding rounds, hiring signals, intent data). Every lead gets a revival score from 1–10 with a plain-English explanation. The calling team sees a ranked list — hottest leads at the top — and knows exactly who to call first and why.

Build strategy

The builder's own sales business is the pilot. Once ReviveIQ recovers 10–15 leads into paying customers internally, that becomes the case study used to sell to other sales teams. The positioning is not "buy more software" — it is "recover revenue from leads you already paid to acquire."

Phase 1 focus (now)

> Lead scoring engine + metrics dashboard. Everything else — warm-up messages, call briefs, proposals — is Phase 2 and beyond. Nail the score first.

Backend split decision

The web layer (auth, CRM connection, dashboard API) runs on **Next.js API Routes** deployed to Vercel. The AI batch processing pipeline runs on a separate **Python FastAPI** service (Railway/Render) because scoring 1,000 leads in parallel will exceed Vercel's serverless timeout. Both services share one **Supabase (Postgres)** database.

02 — PRODUCT VISION

Problem, solution, and what we are not building.

Clarity on scope prevents scope creep.

Current workflow (the pilot business)

1

Cron job runs every 6 months

Copies old/inactive leads from main DB into a separate calling team table. No filtering, no prioritisation — all 1,000+ dumped at once.

2

Calling team contacts leads cold

No brief, no context, no idea who is warm. Agents call in random order. Generic "just checking in" openers. Low pick-up and conversion rates.

3

Interested leads manually re-entered

If someone shows interest, the agent manually adds them back to the main CRM pipeline. High error rate, context lost in transfer.

!

Leads slip through constantly

No watchdog. Leads that were never contacted in a batch are silently ignored until the next 6-month cycle. Hot leads go cold again.

What ReviveIQ fixes in Phase 1

PROBLEM	REVIVEIQ SOLUTION
No prioritisation — 1,000 leads dumped flat	Every lead scored 1–10, ranked by revival likelihood
Agents call cold with no context	Score reason tells agent exactly why this lead is worth calling

PROBLEM	REVIVEIQ SOLUTION
Hot leads buried at bottom of list	Hot tier (8–10) surfaced immediately, visible to manager
No visibility on what was contacted	Dashboard shows full batch status, who was scored, when
Results lost after each batch run	All scores stored in DB — history kept, trends visible over time

Explicit out of scope (Phase 1)

- Automated WhatsApp / email warm-up before calls
- AI-generated call briefs and talking points
- Live call conversation intelligence
- Auto CRM push after a call completes
- Instant proposal generation during a call
- LinkedIn enrichment (requires paid third-party API)
- Website visitor tracking pixel
- Multi-user teams / role-based access within one account
- Mobile app

03 — SYSTEM ARCHITECTURE

Tech stack and how the pieces connect.

Two services, one database. Clear separation between web layer and AI pipeline.

Architecture diagram

Tech stack decisions

LAYER	TOOL	DECISION RATIONALE
Frontend + web API	**Next.js 16 App Router**	Already in use. Vercel deployment. API routes handle auth, CRM connect, dashboard data reads.
Auth	**Clerk**	Fastest integration with App Router. Handles OAuth, magic links, JWT sessions. Free tier covers pilot phase.
Database	**Supabase (Postgres)**	Managed Postgres + row-level security + real-time subscriptions + storage. Free tier sufficient. Single platform.
AI batch pipeline	**Python FastAPI**	No Vercel timeout constraints. Python anthropic SDK more ergonomic for agent orchestration. async/await parallelism for 1,000+ leads.
AI model	**Claude Haiku 4.5**	~$0.001 per lead scored. Fast (1–2s per call). Sufficient reasoning quality for structured scoring rubric. Haiku 4.5 is latest.
Web enrichment	**Tavily**	Already integrated. Live web search designed for AI agents. 1,000 free searches/month covers pilot. 5 parallel searches per lead.
Email	**Resend**	Already wired in codebase. Simple API, generous free tier, excellent deliverability.
Scheduling	**Vercel Cron**	Zero infra overhead. Calls internal Next.js API route weekly which then triggers FastAPI pipeline.
Payments	**Stripe**	Industry standard. Built after pilot validates value — not needed for Phase 1.

04 — END-TO-END USER FLOW

What happens from signup to scored leads.

1

Landing page — reviveiq.com

Visitor sees the marketing page. Clicks "Sign up free" or "Get early access". Redirected to Clerk sign-up screen. Email or Google OAuth.

2

Auth — Clerk

User creates account. Clerk session created. User record written to Supabase `users` table via Clerk webhook. Redirected to dashboard.

3

Dashboard onboarding — connect CRM

First-time user sees prompt to connect their CRM. Selects CRM type, enters API key. Key is AES-256 encrypted and saved to `crm_connections`. ReviveIQ immediately fetches up to 500 inactive leads and stores them in `leads` table.

4

Score batch — Python FastAPI

User clicks "Run scoring". Dashboard calls FastAPI `POST /batch/score`. Pipeline enriches all leads in parallel via Tavily (5 web searches per lead), then scores all leads in parallel via Claude Haiku. Results written to `lead_scores`. Batch status updates in real-time via Supabase subscription.

5

Results dashboard

Dashboard renders ranked lead table from DB. Hot / Warm / Cold metric cards at top. Clicking a lead opens detail panel: score breakdown, web signals found, score reason. Manager can export CSV for the calling team.

6

Weekly automation — Vercel Cron

Every Monday at 6am: sync new dead leads from CRM. 7am: re-score all leads with fresh web intelligence. 8am: send email digest to manager — top 10 hot leads + any leads whose score jumped by 3+ points.

7

Calling team acts

Manager reviews ranked list. Calls in score order. Hot leads (8–10) first, with score reason giving instant context. No more cold calling blind from a flat list.

05 — DATABASE SCHEMA

What gets stored, where, and why.

All tables in Supabase (Postgres). Row-level security enabled — users only see their own data.

crm_connections — one row per CRM connected per user

id PK	uuid	Auto-generated primary key
user_id	text	Clerk user ID — foreign key to auth.users
crm_type	text	hubspot | pipedrive | zoho | salesforce | teamgate
api_key_enc	text	AES-256 encrypted API key — never stored in plaintext
instance_url	text	Salesforce only — org URL like https://org.salesforce.com
last_synced_at	timestamptz	When leads were last pulled from this CRM
created_at	timestamptz	When the connection was first made

leads — every dead lead pulled from CRM or CSV

id PK	uuid	Auto-generated primary key
user_id	text	Clerk user ID — row-level security key
source_crm	text	hubspot | pipedrive | zoho | salesforce | teamgate | csv
external_id	text	Original record ID from the CRM — used for deduplication on sync
name	text	Full name of the lead
contact	text	Email or phone number

company	text	Company name — used as search query for Tavily enrichment
interest	text	What product/service they were interested in
last_contact_date	text	When the last interaction happened
cold_reason	text	Why the lead went cold — critical input for scoring
budget	text	Budget mentioned if any
notes	text	Free-form notes from CRM
raw_data	jsonb	Full original record from CRM — preserved for future use
created_at	timestamptz	When this lead was first imported
updated_at	timestamptz	Last time this record was updated from CRM sync

lead_scores — one row per scoring run per lead (history kept)

id PK	uuid	Auto-generated primary key
lead_id FK	uuid	References leads.id
batch_id FK	uuid	References batches.id — which run produced this score
score	integer	Final revival score 1–10 (base score + web signal boost, capped at 10)
base_score	integer	Score from rubric dimensions before web signal boost
boost	integer	Points added from Tavily web signals (0–3)
score_reason	text	One plain-English sentence explaining the score
signals	jsonb	Full Tavily enrichment output: funding, hiring, intent, competitors, growth
scored_at	timestamptz	When this score was generated — used for freshness display

batches

id PK	uuid

usage

id PK	uuid

user_id	text
status	text
lead_count	integer
scored_count	integer
created_at	timestamptz
completed_at	timestamptz

user_id	text
month	text
leads_scored	integer
plan	text
updated_at	timestamptz

06 — LEAD SCORING MODEL

How every lead gets its revival score.

Deterministic rubric + AI interpretation. Transparent, consistent, explainable.

Design principle

The score is not a black box. Claude does not invent the number — it interprets qualitative text (cold reason, notes) and maps it to pre-defined dimension buckets. The math is deterministic. This means scores are consistent across runs and easy to explain to a sales manager.

Scoring dimensions

DIMENSION 1 — RECENCY (MAX 3 PTS)		
Went cold	3–6 months ago	+3
	6–12 months ago	+2
	1–2 years ago	+1
	2+ years ago	+0
DIMENSION 2 — COLD REASON (MAX 2 PTS)		
Budget / timing	Temporary constraint — budget next quarter, team restructuring, etc.	+2
Lost to competitor	Chose a competitor — contracts end, dissatisfaction possible	+1
Not interested	Explicit rejection, ghosted, not a fit	+0

DIMENSION 3 — BUDGET SIGNAL (MAX 2 PTS)		
Budget specified	A specific budget amount was mentioned in CRM notes	+2
Budget unknown	No budget mentioned	+1

DIMENSION 4 — ENGAGEMENT HISTORY (MAX 1 PT)		
High engagement	Multiple touchpoints, attended demo, was close to closing	+1
Low engagement	Single contact, cold inquiry, no demo	+0

WEB SIGNAL BOOST — TAVILY (MAX +3 PTS ON TOP OF BASE)		
Funding detected	Company raised money recently — more budget available	+2
Active intent	Company evaluating similar tools right now	+2
Hiring signals	Hiring for roles that need this product	+1
Competitor activity	Reviewed G2/Capterra for similar products	+1
Growth signals	Company expanding — new office, growing team	+1

Score calculation

Base score = Recency + Cold Reason + Budget + Engagement (max 8). Web boost = min(sum of Tavily signals, 3). Final score = min(base + boost, 10). Minimum score = 1.

Score tiers and actions

TIER	SCORE	COLOUR	RECOMMENDED ACTION
Hot	8–10	Green	Call this week — clear buying signal or timing window
Warm	5–7	Yellow	Call this month — viable, worth the effort
Cold	1–4	Red	Low priority — contact only after hot and warm list cleared

Claude's prompt contract

Claude Haiku receives the lead record, the enrichment signals, and the rubric. It must return JSON only with these fields:

```
{
  "recency_score":     2,     // 0–3
  "reason_score":      2,     // 0–2
  "budget_score":      1,     // 0–2  (1 = unknown, 2 = mentioned)
  "engagement_score":  1,     // 0–1
  "score_reason":      "Went cold due to budget constraint 8 months ago; had a dem
}
```

The API route applies the boost and computes the final score. Claude never outputs the final score — it only fills the rubric dimensions.

07 — AI AGENTS DESIGN

Three agents. One pipeline.

Each agent has a single clear job. No agent does more than one thing.

🔍 Agent 1 — Enrichment Agent

Trigger: called once per lead during batch run

Tavily

INPUT	Lead name, company name, interest/product category
PROCESS	Runs 5 Tavily searches simultaneously (asyncio.gather): 1. `"{company}" funding raised investment 2024 2025` 2. `"{company}" hiring job {product} OR "sales operations" 2025` 3. `"{company}" evaluating OR looking for {product} OR switching` 4. `"{company}" review G2 Capterra {product} OR similar` 5. `"{company}" expansion growth new office team 2025`
OUTPUT	Signals object: { funding_raw, hiring_raw, intent_raw, competitor_raw, growth_raw } — raw text from search results
COST	5 Tavily API calls per lead. Free tier: 1,000 searches/month = 200 leads/month enriched free
CACHING	Signals stored in lead_scores.signals column. Not re-fetched unless it's a weekly re-score run

🧠 Agent 2 — Scoring Agent

Trigger: called per lead after enrichment completes

Claude Haiku 4.5

INPUT	Lead record (name, contact, interest, last_contact_date, cold_reason, budget, notes) + enrichment signals object

PROCESS	System prompt contains the full scoring rubric. User message contains the lead data. Claude maps qualitative fields to rubric dimensions and returns dimension scores + score_reason as JSON only. The API applies boost math and computes final score.
OUTPUT	{ recency_score, reason_score, budget_score, engagement_score, score_reason } — Claude fills the rubric, not the final number
COST	~$0.001 per lead at Haiku pricing. 1,000 leads = ~$1 total
SPEED	1–2 seconds per lead. 1,000 leads in parallel ≈ 10–15 seconds total on FastAPI

Agent 3 — Batch Orchestrator

Trigger: manual (user clicks "Run scoring") or Vercel Cron weekly

FastAPI

INPUT	user_id, optional: list of specific lead IDs to score (else scores all)
PROCESS	1. Check usage table — block if monthly limit exceeded 2. Create batch record in DB (status: running) 3. Pull all leads for user from Supabase 4. Run Agent 1 on all leads in parallel (asyncio.gather) 5. Run Agent 2 on all leads in parallel with their signals 6. Write all scores to lead_scores table 7. Increment usage.leads_scored 8. Update batch status → done 9. If triggered by cron: call /notify/digest endpoint
OUTPUT	Batch record updated in DB. Dashboard reads scores via Supabase real-time subscription — no polling needed.
ENDPOINT	POST /batch/score — accepts { user_id, lead_ids? } — returns { batch_id, status }

08 — CRM INTEGRATION

How dead leads get into the system.

Supported CRMs

CRM	AUTH METHOD	WHAT IS PULLED	FILTER
HubSpot	Private App Token	Contacts	lead_status = cold, unqualified, or not last contacted in 90+ days
Pipedrive	API Token	Persons	last_activity_date older than 90 days
Zoho CRM	OAuth Access Token	Leads module	Lead_Status = Cold, Inactive, Not Contacted
Salesforce	Access Token + Instance URL	Lead object	IsConverted = false, LastActivityDate oldest first
Teamgate	API Key	Leads	status = inactive

Sync strategy

EVENT	WHAT HAPPENS
First connect	Pull up to 500 inactive leads immediately. Store in `leads` table with `external_id` for deduplication.
Weekly cron (Monday 6am)	Pull records updated or created since `last_synced_at`. Insert new, update changed. Update `crm_connections.last_synced_at`.
Manual re-sync	User can trigger from Settings page at any time. Same logic as weekly cron.

Security — API key storage

CRM API keys are encrypted with AES-256 before being written to the database. The encryption key is stored in an environment variable, never in the DB. Keys are decrypted

> only at the moment they are used (inside the FastAPI service) and never returned to the frontend.

Deduplication

Each lead record stores the original CRM record ID as `external_id`. On every sync, the pipeline uses an upsert (INSERT … ON CONFLICT (user_id, source_crm, external_id) DO UPDATE) so duplicate imports never create duplicate rows.

CSV fallback

For users without a supported CRM, CSV upload remains available. Required columns: `name, contact, interest, lastContact, reason`. Optional: `budget, notes`. CSV leads get `source_crm = "csv"` and no `external_id`.

09 — SCHEDULING & DATA FRESHNESS

How scores stay current without manual effort.

Weekly cron schedule (Vercel Cron)

TIME	JOB	WHAT IT DOES
Monday 6:00am	CRM sync	Pulls new dead leads from all connected CRMs for all users. Inserts/updates `leads` table.
Monday 7:00am	Re-score	Calls FastAPI `POST /batch/score` for each user. Fresh Tavily searches. All leads re-scored. New rows in `lead_scores`.
Monday 8:00am	Email digest	Queries top 10 hot leads per user. Checks for score jumps (≥3 points). Sends digest email via Resend.

Freshness display

Every score row has a `scored_at` timestamp. The dashboard shows "Scored 2 days ago" next to each lead. If a score is older than 14 days, it shows a warning badge nudging the user to re-run scoring.

Manual trigger

Users can click "Run scoring" from the dashboard at any time. This is a direct call to the FastAPI batch endpoint — no cron involved. Usage is deducted from their monthly allowance.

> **Why weekly re-score matters**
>
> A lead that scored 4 last month (cold — budget constraint) might score 9 this week if Tavily finds they just raised a funding round. Without re-scoring, the calling team would never know to call them. Weekly cadence keeps the ranked list accurate to current reality.

Score jump alert (immediate)

Separate from the weekly digest. Every time a re-score completes, the pipeline compares new scores to previous scores. If any lead's score increased by 3 or more points, an immediate email alert is sent — regardless of what day it is. This is the "hot signal" notification.

10 — NOTIFICATIONS

How sales managers get informed without noise.

Design principle

The dashboard is the source of truth — not email. Email only notifies managers when something worth acting on has changed. No daily spam, no full lead dumps over email.

Weekly digest email — every Monday

FIELD	CONTENT
Subject	"Your top leads this week — ReviveIQ"
Body	Top 10 hot leads (score 8–10) with: name, score, one-line reason, last contact date
Score changes	Any leads whose score jumped since last week, highlighted with old → new score
CTA	Single button: "Open dashboard" — no full list in email, just a preview
Sent via	Resend (already integrated)

Score jump alert — immediate

FIELD	CONTENT
Trigger	Lead score increases by ≥ 3 points in any re-score run
Subject	"🔥 Hot signal: [Lead name] jumped from 4 → 9"
Body	Lead name, old score, new score, what changed (e.g. "Just raised Series A funding")
CTA	"View lead" — deep link to lead detail in dashboard

FIELD	CONTENT
Sent via	Resend

In-app notifications (future)

Not in Phase 1. Phase 2 can add a notification bell in the dashboard using Supabase real-time subscriptions — when a batch completes or a score jumps, the dashboard updates without page refresh.

What is NOT sent over email

- Full lead list with all scores — that lives in the dashboard
- Cold or warm leads (1–7) — only hot leads and score jumps are worth an alert
- Daily emails — weekly cadence prevents notification fatigue
- Individual lead WhatsApp messages or call briefs — Phase 2

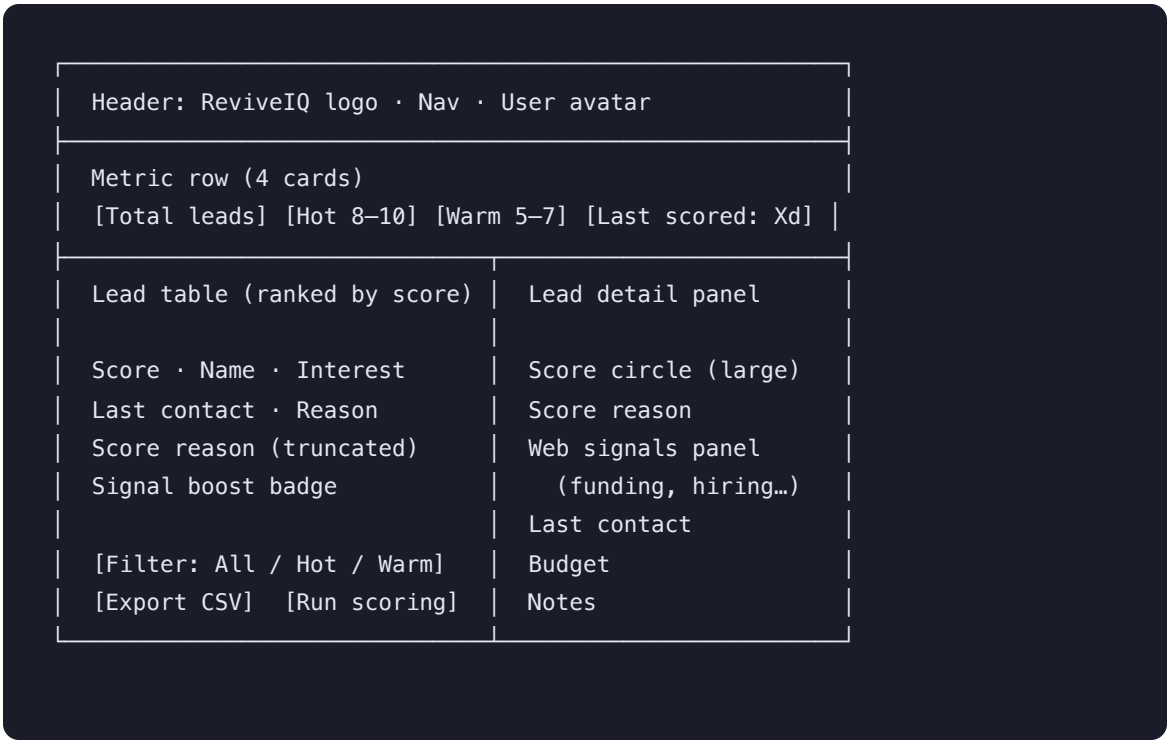
11 — DASHBOARD DESIGN

What the sales manager sees every day.

Routes

ROUTE	AUTH	DESCRIPTION
/	Public	Marketing landing page with waitlist form
/sign-in	Public	Clerk sign-in UI
/sign-up	Public	Clerk sign-up UI
/dashboard	Protected	Main lead scoring view — ranked table + metric cards
/dashboard/settings	Protected	CRM connections, notification email, plan & usage

Dashboard main view — component layout



Lead table columns

COLUMN	DISPLAY	SORTABLE
Score	Coloured circle — green (8–10), yellow (5–7), red (1–4)	Yes — default sort
Name	Full name	No
Contact	Email or phone	No
Interest	Product/service they wanted	No
Last contact	Relative (e.g. "8 months ago")	Yes
Cold reason	Truncated to 40 chars	No
Signals	Coloured dot if web boost > 0	No
Scored	"3 days ago" — freshness indicator	Yes

Settings page

- CRM connections: connect / disconnect, shows connection status and last synced timestamp
- Notification email: where weekly digest and alerts are sent
- Plan & usage: progress bar showing X / 1000 leads scored this month, upgrade CTA
- Manual re-sync: trigger a CRM pull from settings without going through the dashboard

12 — MONETISATION MODEL

Usage-based pricing tied to recovered revenue.

Plans

PLAN	LEADS SCORED / MONTH	CRM CONNECTIONS	EMAIL ALERTS	PRICE
Free	100	1	Weekly digest only	₹0
Pro	1,000	3	Digest + score jump alerts	TBD after pilot
Scale	Unlimited	Unlimited	All alerts + custom frequency	TBD after pilot

Pricing strategy note

Do not set prices before the pilot proves value. Run the pilot on Free tier. Once 10–15 internal leads convert to customers, calculate the recovered revenue. Price Pro at roughly 10% of the value of one recovered deal per month. That makes the ROI obvious and the upgrade an easy decision.

Usage tracking logic

- Before every batch run: check `usage` table for current month's `leads_scored`
- If `leads_scored + batch_size > plan_limit` : block the run and show upgrade prompt
- If within limit: run batch, increment `leads_scored` by actual leads processed
- Usage resets on the 1st of each calendar month (new row in `usage` table)
- Re-scoring existing leads counts toward the monthly total — incentivises paid plans for weekly automation

Stripe integration (Phase 4 — after pilot)

- Upgrade button triggers Stripe Checkout session
- Stripe webhook on payment success updates `usage.plan` in DB
- Downgrade at period end — plan reverts to Free, usage cap drops
- No credit card required for Free tier

13 — PHASED BUILD PLAN

What gets built, in what order, and why.

Each phase delivers standalone value before the next begins.

1 Foundation — Persistence & Auth

Start here

- → Install Supabase JS client + set up project, create all 5 tables with row-level security
- → Install Clerk, wrap app in ClerkProvider, add middleware to protect /dashboard routes
- → Save CRM leads to DB on connect (currently dropped after session ends)
- → Save scored results to DB (currently lost on page refresh)
- → Dashboard reads leads + scores from DB instead of local state
- → Set up .env.local with SUPABASE_URL, SUPABASE_ANON_KEY, CLERK_PUBLISHABLE_KEY, CLERK_SECRET_KEY

2 Python FastAPI Scoring Service

After Phase 1

- → Create FastAPI project with /batch/score and /batch/sync-crm endpoints
- → Port enrichment agent (Tavily) from Next.js API route to Python async functions
- → Restructure scoring agent to use the rubric model — Claude fills dimensions, API computes final score
- → Batch orchestrator: parallel enrichment + scoring, write to Supabase, update batch status
- → Deploy to Railway or Render
- → Update Next.js dashboard to call FastAPI instead of /api/process-leads

3 Automation & Notifications

After Phase 2

- → Vercel Cron: weekly CRM sync job (Monday 6am)
- → Vercel Cron: weekly re-score job (Monday 7am) — calls FastAPI
- → Weekly email digest via Resend — top 10 hot leads sent Monday 8am
- → Score jump alert — immediate email when any lead score jumps ≥ 3 points
- → Usage tracking — leads_scored counter incremented per batch
- → Usage gate — block batch run if monthly limit exceeded, show upgrade prompt

4 Monetisation

After pilot proves value

- → Stripe Checkout integration — upgrade button triggers payment flow
- → Stripe webhook — updates user plan in DB on successful payment
- → Plan management UI in Settings page
- → Downgrade handling — revert plan at period end

5 Calling AI Features

Future — not now

- → Automated personalised WhatsApp / email warm-up before calls
- → AI-generated call brief with talking points, objections, what to avoid
- → Auto CRM push — push revived lead back to pipeline after successful call
- → Conversation intelligence — live cheat sheet during the call
- → Instant proposal generation — personalised proposal in 60 seconds on the call

→ Slip-through watchdog — flag leads never contacted in a batch

14 — WHAT IS ALREADY BUILT

Starting point — what exists today.

Built as of April 17, 2026. This is the working foundation Phase 1 builds on.

Landing page — `src/app/page.tsx`

- Full ReviveIQ marketing page: hero, stats bar, problem/solution comparison, how-it-works, AI signals section, CRM logo strip
- Early access waitlist form — email + company → POST /api/waitlist
- Design system: #080808 dark background, #00e5a0 teal accent, Geist serif headings — matches brand spec
- Floating demo card with sample scored leads and score circles

Dashboard — `src/app/dashboard/page.tsx`

- 2-step flow: business setup (company name, product, tone) → lead import
- CRM connect toggle: select CRM, paste API key, click Connect → fetches leads live
- CSV upload: paste raw CSV with sample data preloaded
- Processing screen with progress bar
- Results view: hot/warm/cold metric cards, ranked lead list with score circles, detail panel with web signals + score reason
- Export to CSV button
- **Missing** : no auth, results not persisted to DB, lost on refresh

API Routes

ROUTE	STATUS	DOES WHAT
`/api/crm`	Working	Connects to HubSpot, Pipedrive, Zoho, Salesforce, Teamgate and fetches leads
`/api/process-leads`	Working	Enriches with Tavily + scores with Claude Haiku + returns ranked results
`/api/waitlist`	Partial	Accepts signups. Sends to Google Sheets + Resend only if env vars are set
`/api/enrich-lead`	Exists	Standalone enrichment endpoint — not wired to dashboard UI yet

UI components

shadcn/ui: Badge, Card, Input, Progress, Select, Separator, Table, Textarea, Button. Tailwind CSS v4. IBM Plex Sans font. Custom CSS variables for design tokens (--teal, etc.).

Dependencies installed

`@anthropic-ai/sdk`, `@tavily/core`, `resend`, `next 16.2.3`, `react 19`, `tailwindcss v4`, `shadcn/ui`

Not yet installed: `@supabase/supabase-js`, `@supabase/ssr`, `@clerk/nextjs`

15 — OPEN DECISIONS & RISKS

What needs a decision before building.

Open decisions

DECISION	OPTIONS	RECOMMENDATION
Python backend hosting	Railway, Render, Fly.io, VPS	**Railway** — simplest deploy from GitHub, free starter tier, $5/month hobby plan for always-on
API key encryption	AES-256 in app, Supabase Vault, HashiCorp Vault	**AES-256 in app** for Phase 1 — simple, sufficient, upgrade to Vault later if needed
Real-time batch progress	Supabase real-time, polling, Server-Sent Events	**Supabase real-time** — dashboard subscribes to `batches` table, updates automatically as `scored_count` increments
Pricing numbers	Set now vs. after pilot	**After pilot** — build the infrastructure now but leave prices TBD until recovered revenue is proven

Risks and mitigations

RISK	LIKELIHOOD	MITIGATION
Tavily free tier exhausted (1,000 searches/month = 200 leads)	Medium — happens as soon as pilot grows	Tavily Pro is $20/month for 10,000 searches. Build in usage tracking so we know when we're close. Factor into pricing.
CRM API tokens expire (especially Zoho OAuth)	High for Zoho	Detect auth errors on sync, email user to reconnect. Add token refresh flow for Zoho in Phase 2.
Claude Haiku scores inconsistently on	Medium	Rubric reduces ambiguity significantly. Add a few-shot examples to the system prompt

RISK	LIKELIHOOD	MITIGATION
ambiguous cold reasons		if consistency issues are observed in pilot.
Vercel Cron misses a run (rare but possible)	Low	Cron is best-effort on Vercel. For the pilot this is acceptable. If reliability becomes critical, move cron to the FastAPI service with its own scheduler.
User's CRM has non-standard "inactive" lead status values	Medium	Pull broadly (all contacts/leads with no activity in 90+ days) rather than filtering by status field, which varies by CRM setup.

First actions to take right now

- Create Supabase project → get Project URL and anon key
- Create Clerk application → get Publishable key and Secret key
- Share those 4 keys so Phase 1 build can begin immediately
- Decide on Railway vs Render for FastAPI hosting (Railway recommended)

Summary verdict

Next.js + Clerk for the web layer. Python FastAPI on Railway for the AI batch pipeline. Supabase as the shared database. Claude Haiku fills a deterministic scoring rubric. Tavily provides live web intelligence. Resend sends weekly digests and score alerts. Stripe handles payments after the pilot. Build in that order.